

LKPD 2

Deploy Aplikasi PHP dan MySQL (Multi-Container)

Repository aplikasi: <https://github.com/paknux/apptoko.git>

A. Persiapkan Docker

1. Instal paket yang dibutuhkan

```
$ sudo apt update && sudo apt install -y git nano curl links mc docker.io nmap  
$ sudo usermod -aG docker nugroho  
$ sudo newgrp docker
```

2. Pull image yang diperlukan

```
$ docker pull ubuntu:24.04  
$ docker pull mariadb:11-jammy
```

Lihat hasilnya:

```
$ docker image ls
```

3. Buat jaringan yang akan digunakan untuk komunikasi antar container

```
$ docker network create mynet
```

4. Persiapkan folder untuk dimounting ke DocumentRoot webserver

```
$ sudo su  
# mkdir -p /var/mywww  
# cd /var/mywww  
# git clone https://github.com/paknux/apptoko.git .
```

5. Buat file konfigurasi koneksi database aplikasi

```
# nano config.php
```

Isi dengan:

```
define('DB_HOST', 'dbserver');  
define('DB_USER', 'root');  
define('DB_PASS', 'pass123');  
define('DB_NAME', 'toko_db');
```

Catatan: Nilai DB_HOST diisi dengan nama container database (dbserver), bukan localhost atau IP. Ini karena container webserver dan dbserver nanti dijalankan pada network Docker yang sama (mynet), sehingga saling terhubung menggunakan nama container sebagai hostname.

B. Jalankan Container MariaDB (Database Server)

1. Jalankan container image mariadb:11-jammy tanpa mengekspose port ke host. Ini lebih aman karena database hanya bisa diakses oleh container lain yang berada pada network yang sama (mynet).

```
docker run -d --name dbserver --network mynet -e MYSQL_ROOT_PASSWORD=pass123
mariadb:11-jammy
```

(Opsional) Jika memang butuh mengakses database dari luar (misal dari komputer host, atau tools seperti DBeaver/HeidiSQL), tambahkan parameter -p:

```
docker run -d --name dbserver --network mynet -p 3306:3306 -e
MYSQL_ROOT_PASSWORD=pass123 mariadb:11-jammy
```

2. (Opsional) Buat database yang dibutuhkan aplikasi

```
docker exec -it dbserver mysql -u root -ppass123 -e "CREATE DATABASE toko_db;"
```

(Opsional) Jika ada file dump/skema SQL bawaan aplikasi, import dengan:

```
docker exec -i dbserver mysql -u root -ppass123 toko_db < /var/www/toko_db.sql
```

C. Buat Image Ubuntu 24.04 dengan Apache + PHP

Tahap ini membangun image kustom secara bertahap (v1 → v2) menggunakan docker commit, dimulai dari image dasar ubuntu:24.04.

1. Buat container sementara dari image ubuntu:24.04, dibiarkan tetap hidup dengan tail -f /dev/null supaya bisa dimasuki untuk instalasi manual

```
docker run -d --name ubuntu ubuntu:24.04 tail -f /dev/null
```

2. Masuk ke dalam container yang sedang berjalan

```
docker exec -it ubuntu bash
```

3. Di dalam container, install Apache dan PHP beserta modul yang dibutuhkan

```
apt update
DEBIAN_FRONTEND=noninteractive apt install -y nano apache2 php php-mysqli php-mysql
libapache2-mod-php mysql-client curl
```

4. Uji jalankan Apache untuk memastikan instalasi berhasil, lalu hentikan kembali

```
apache2ctl start
apache2ctl stop
exit
```

Catatan: Apache2 dalam container ubuntu memang tidak dapat diakses/ditests dari luar menggunakan browser, karena tidak ada port yang diekspose.

5. Simpan kondisi container ini menjadi image baru (ubuntu:v1), lalu hapus container sementara

```
docker commit ubuntu ubuntu:v1
docker rm ubuntu -f
```

6. Uji image ubuntu:v1 dengan menjalankan Apache secara foreground sebagai proses utama container (CMD), supaya container tetap hidup selama Apache berjalan

```
docker run -d --name ubuntu1 -p 81:80 ubuntu:v1 apache2ctl -D FOREGROUND
```

Cek apakah Apache berhasil diakses:

```
curl http://localhost:81
```

7. Jika sudah berjalan dengan baik, simpan sebagai image final (ubuntu:v2) dan bersihkan container ujinya

```
docker commit ubuntu1 ubuntu:v2
docker rm ubuntu1 -f
```

D. Jalankan Container Web Server (Final)

Jalankan container web server menggunakan image ubuntu:v2, dihubungkan ke network mynet (agar bisa mengakses dbserver), dan folder aplikasi di-mount ke DocumentRoot Apache.

```
docker run -d \
  --name webserver \
  --network mynet \
  -p 8088:80 \
  -v /var/mywww:/var/www/html \
  --restart unless-stopped \
  ubuntu:v2 \
  apache2ctl -D FOREGROUND
```

Penjelasan parameter:

Parameter	Fungsi
--network mynet	Menghubungkan container ke network yang sama dengan dbserver, sehingga bisa saling akses menggunakan nama container
-p 8088:80	Meneruskan port 8088 di host ke port 80 (Apache) di dalam container
-v /var/mywww:/var/www/html	Mount folder aplikasi dari host ke DocumentRoot Apache, sehingga perubahan file di host langsung terlihat tanpa build ulang image
--restart unless-stopped	Container otomatis menyala kembali jika Docker/host di-restart

apache2ctl -D FOREGROUND	Menjalankan Apache sebagai proses utama (PID 1) container, sehingga container tetap hidup selama Apache berjalan
-----------------------------	--

E. Verifikasi

1. Cek semua container sudah berjalan

```
docker ps
```

Pastikan dbserver dan webserver berstatus Up.

2. Cek aplikasi bisa diakses

```
curl http://localhost:8088
```

Atau buka lewat browser: `http://<IP-server>:8088`

3. Cek koneksi webserver ke dbserver (dari dalam container)

```
docker exec -it webserver bash -c "mysql -h dbserver -u root -ppass123 -e 'SHOW DATABASES;'"
```

F. Perintah Bantuan (Troubleshooting)

Kebutuhan	Command
Lihat log container	<code>docker logs -f webserver</code>
Masuk ke shell container yang jalan	<code>docker exec -it webserver bash</code>
Restart container	<code>docker restart webserver</code>
Lihat network & container yang tergabung	<code>docker network inspect mynet</code>
Hapus semua container (reset total)	<code>docker rm -f webserver dbserver</code>

Catatan: Password "pass123" pada dokumen ini hanya untuk keperluan pembelajaran/lokal. Untuk lingkungan produksi, gunakan password yang kuat dan pertimbangkan menyimpan kredensial melalui Docker secret atau file .env, bukan langsung di command line.